WEEK 8 • GUIDED GRADUATE LABORATORY

Kernel-scoped CSR SpMV: from C++ to memory-system evidence

gem5 25.1.0.0 / Ramulator 2.1 / Verilator / Python

One original Week 7 kernel; two controlled locality patterns; two DRAM models; one explicit verification contract.

Workload	Platform and scope
Original functional baseline	Week 7 C++17 CSR SpMV, unchanged and numerically verified
Large experimental cases	32,768 rows × 16 nonzeros/row = 524,288 nonzeros
Compute model	gem5 x86 timing CPU + L1 caches + L2 + DDR4 reference
Trace and DRAM	Post-L2 request trace → Ramulator DDR4 / HBM2 replay
RTL counterpart	Verilator six-contribution ready/valid accumulator check
Analysis	Python: request conservation, byte equivalence, modeled time, caveats

This handout is the classroom replacement for a slide presentation. Students read the code, make predictions, run experiments in their own directories, and defend results with explicit assumptions.

No student needs to install gem5, Ramulator, Python packages, or Verilator. The instructor's copies are used read-only. Do not log in as the instructor, request their password, use sudo, or write into their software directories.

Learning outcomes

- Identify exactly which C++ instructions belong to the measurement region and which do not.
- Explain how gem5 represents CPU timing, caches, main memory, ROI events, and packet traces.
- Convert trace commands and validate request/byte conservation before DRAM replay.
- Compare DDR4/HBM2 using byte-equivalent requests and physical-clock normalization.
- Explain an RTL ready/valid stall and relate protocol correctness to architecture modeling.
- Separate measured model output from causal interpretations and unsupported speedup claims.

Recommended 180-minute class

Phase	Minutes	Student activity
Orient and predict	20	Access check, CSR baseline and predicted locality effect
Study ROI + gem5	40	C++ markers; caches; stats; inspect real ROI outputs
Run paired gem5	35	Scattered and clustered case; trace/manifest
DRAM model and normalization	35	Ramulator replay; 64B/32B accounting
RTL protocol laboratory	20	Verilator handshake and backpressure
Analyze and defend	30	Four-way comparison, alternatives, evidence submission

NOTE Instructor: the already completed large gem5 experiments took ~233–234 host seconds each when run on server1. Concurrent student runs may be slower; preload verified results as a fallback. Host elapsed time is not simulated kernel time.

1. What is supplied and what is installed?

The classroom bundle has six files. The installer/build steps already completed on server1 are not repeated by students.

File	Role	Student modification?
week7_spmv_lab.cpp	Unchanged Week 7 CSR oracle and analytical sweep	No
Week8_Kernel_Only_CSR_SpMV.py	Generates C++ ROI harness and gem5 configuration; runs kernel-scoped trace and normalized first replay	Parameter changes only
Week8_Ramulator2_DDR4_HBM2_Replay.py	Ramulator front-end and controller model with original LD/ST format	Read only
Week8_Classroom_Payload_Replay.py	Classroom adaptation: replay 64B DDR4 / 2×32B HBM2 into student results	Parameter changes only
Week8_Verilator_CSR_Protocol_tb.sv	RTL ready/valid accumulator and testbench for six Week 7 contributions	Edit copy for extension only
README.txt	Release notes; scope and access assumptions	No

Installed shared tools — read-only

Component	Instructor installation path
gem5 source/binary	/home/asinha/gem5 → build/X86/gem5.opt
gem5 M5ops library	/home/asinha/gem5/util/m5/build/x86/out/libm5.a
Ramulator 2.1 source	/home/asinha/tools/week8/ramulator2
Python 3.11 virtual environment	/home/asinha/tools/week8/venv/bin/python
Verilator	/usr/bin/verilator + system verilator-devel package
Modern GCC	/opt/rh/gcc-toolset-13/enable

NOTE The Ramulator installation includes a local callback-based drain patch, marked W8_DRAIN_CALLBACK_PATCH_V1. It is not the unmodified upstream source. Students must not rebuild or patch the instructor's installation.

Instructor-only: prepare the shared classroom directory

Complete this once from the instructor account before releasing the document. From your Mac, upload the matching source ZIP, then on server1 extract it into the approved read-only course directory:

```
scp ~/Downloads/Week8_Student_CSR_SpMV_Source_Bundle.zip asinha@server1.local:/tmp/

COURSE=/home/asinha/data/data-intensive/week8/classroom
mkdir -p "$COURSE"
unzip -o /tmp/Week8_Student_CSR_SpMV_Source_Bundle.zip -d "$COURSE"
sha256sum "$COURSE/week7_spmv_lab.cpp"
# Expected SHA-256:
# d5c32d51a698aa4619d7e515a4fa66f2813e8b12ddf2ae34fefd6cfc3ef962b5
```

Ask your system administrator to give the course Unix group read/traverse access only to the approved classroom directory and required simulator paths. Do not make the entire instructor home directory public. Check the setup using a normal student account before class.

If a user cannot access /home/asinha/... paths, do not share the instructor's credentials. Fix permissions through a dedicated course group or an administrator-managed read-only software installation.

2. Student setup: run only in your own account

SSH to server1 using your own username. The following commands do not need sudo and do not modify the installed simulators.

```
source /opt/rh/gcc-toolset-13/enable
export ROOT="$HOME/data/data-intensive/week8"
export LAB="$ROOT/lab"
export COURSE=/home/asinha/data/data-intensive/week8/classroom
export PY=/home/asinha/tools/week8/venv/bin/python
export GEM5_ROOT=/home/asinha/gem5
export RAM_ROOT=/home/asinha/tools/week8/ramulator2
mkdir -p "$LAB" "$ROOT/results"
cp "$COURSE"/*.py "$COURSE"/*.cpp "$COURSE"/*.sv "$LAB"/
```

Verify access without attempting an installation

```
test -x "$GEM5_ROOT/build/X86/gem5.opt" && echo GEM5_ACCESS=PASS
test -r "$GEM5_ROOT/util/m5/build/x86/out/libm5.a" && echo M5OPS_ACCESS=PASS
"$PY" -c 'import ramulator,yaml; print("RAMULATOR_IMPORT=PASS")'
verilator --version
sha256sum "$LAB/week7_spmv_lab.cpp"
```

NOTE Important: gem5.opt does not accept a --version flag on this installation. To inspect version information, run it under a configuration and read its log, or inspect the source checkout with git.

If a permission check fails, stop. The lab is designed for separate student accounts—not a shared writeable login.

3. Functional baseline: the original CSR computation

CSR representation and the exact Week 7 reference

```
const Ints ROW_PTR{0, 2, 3, 5, 6};
const Ints COL_IDX{0, 3, 1, 0, 4, 2};
const Ints VALUES{2, 1, 3, 4, -1, 5};
const Ints X{1, 2, 3, 4, 5};
const Ints EXPECTED{6, 6, -1, 15};
```

row_ptr[r] and row_ptr[r+1] delimit each row's nonzeros; col_idx[k] chooses the input-vector element; values[k] is the matrix nonzero. The original program computes exactly one output per row.

```
for (size_t row = 0; row < y.size(); ++row)
    for (int k = row_ptr[row]; k < row_ptr[row + 1]; ++k)
        y[row] += values[k] * x[col_idx[k]];
```

Example: row 0 uses 2×x[0] + 1×x[3] = 2×1 + 1×4 = 6. The program validates its own output against a dense reference and the expected four-element vector.

ANALYZE If the number of rows doubles while the average nonzeros per row stays constant, which data structures increase linearly? Which data accesses might grow nonlinearly in latency?

Large benchmark: only the workload scale changes

Parameter	Setting	Derivation
Rows	32,768	row_ptr has 32,769 offsets
Nonzeros per row	16	Uniform row length in this teaching matrix
Total nonzeros	524,288	32,768 × 16
Input-vector length	262,144	4-byte signed integer elements
Arithmetic	Integer multiply + add	Same accumulation statement as Week 7
Pattern	scattered / clustered	Different column index distribution

The new C++ file is generated by Week8_Kernel_Only_CSR_SpMV.py. It preserves the original source by including it with a renamed main function, and compares the new output with the original csr_spmv() after the ROI.

```
#define main week7_preserved_main
#include "week7_spmv_lab.cpp"
#undef main
...
Ints original = csr_spmv(row_ptr, col_idx, val, x);
require(original == y, "W8_LARGE_CSR_ORACLE=FAIL");
```

Why this matters: a performance result is invalid if a faster variant computes different or incorrect output. Both access patterns are separately checked against the Week 7 algorithm, not against each other.

Dataset generator: controlled locality difference

```
uint32_t h = mix32((uint32_t)k ^ 0x9e3779b9u);
col_idx[(size_t)k] = (int)(h % (uint32_t)(
    pattern == "clustered" ? 2048 : cols));
val[(size_t)k] = (int)(k % 5) - 2;
```

mix32 produces a deterministic mixed 32-bit integer for each k. In clustered mode, x indices lie within 2,048 elements = 8 KiB. In scattered mode, indices can span 262,144 elements = 1 MiB. All rows still have 16 nonzeros and the same values sequence.

These are different matrices because their col_idx arrays differ. They are matched in size and arithmetic count, not identical mathematical inputs. This controlled difference isolates a meaningful locality hypothesis but does not prove a universal effect across all sparse matrices.

ANALYZE Before running the simulator, predict whether L1, L2, or DRAM traffic should differ more between the two patterns. State one reason your prediction might fail.

4. Mark the kernel and exclude setup from measurement

Read the actual C++ ROI code

```
// Allocate and initialize row_ptr, col_idx, val, x and y.
validate_csr(row_ptr, col_idx, val, x); // BEFORE ROI
m5_work_begin(0, 0);
csr_spmv_kernel_only(row_ptr, col_idx, val, x, y);
m5_work_end(0, 0);
Ints original = csr_spmv(row_ptr, col_idx, val, x); // AFTER ROI
require(original == y, "W8_LARGE_CSR_ORACLE=FAIL");
```

m5_work_begin and m5_work_end are special gem5 operations. When the binary executes under gem5, they generate work-item events. The gem5 configuration uses the events to reset and dump statistics at the chosen boundaries. The C++ kernel itself contains only nested CSR loops and an integer multiply-accumulate.

In the native-only check, a compile-time switch W8_NATIVE_TEST defines these M5 functions as no-ops. This prevents attempting gem5 magic instructions on the physical server CPU.

```
#ifdef W8_NATIVE_TEST
static inline void m5_work_begin(uint64_t,uint64_t) {}
static inline void m5_work_end(uint64_t,uint64_t) {}
#else
#include <gem5/m5ops.h>
#endif
```

The script compiles one native test binary and one gem5 guest binary. The guest links libm5.a from /home/asinha/gem5/util/m5/build/x86/out. Both binaries use the same CSR computation and inputs.

What “kernel-only” does and does not mean

- Outside the ROI: vector/matrix allocation, deterministic data generation, CSR structural validation, original reference computation, checksum, and printing.
- Inside the ROI: calls into the accumulation function, loop/control instructions, loads/stores for the CSR arrays and output, and associated cache/memory effects.
- Not perfectly attributable: outstanding requests from before the start tick, request/response crossings near the end tick, and instruction traffic sharing the post-L2 path.
- The measured CPU is gem5 X86TimingSimpleCPU, not an FPGA accelerator and not an out-of-order high-end Xeon model.

ANALYZE An instruction load fetches a cache line during the ROI. Should it be removed from the post-L2 memory trace? Why is exact removal difficult with this probe?

5. How the gem5 simulation is constructed

System, clock, CPU and memory mode

```
system = System()
system.clk_domain = SrcClockDomain(clock='2GHz',
                                   voltage_domain=VoltageDomain())
system.mem_mode = 'timing'
system.mem_ranges = [AddrRange('512MiB')]
system.exit_on_work_items = True
system.cpu = X86TimingSimpleCPU()
```

2 GHz defines the modeled CPU clock domain; timing mode enables cycle-sensitive request timing; 512 MiB is the simulated address space. X86TimingSimpleCPU is an in-order timing CPU. exit_on_work_items tells gem5 to return control to its Python driver when the guest triggers a workbegin/workend event.

This is a simplified teaching configuration, not a calibrated physical server model. The DDR4_2400_8x8 device model has a nominal capacity larger than the assigned 512 MiB memory range; gem5 may warn about the mismatch. Do not claim capacity-specific performance effects from it.

Two cache levels and a shared memory bus

```
system.cpu.icache = cache('32KiB', 4, 2, 4)
system.cpu.dcache = cache('32KiB', 8, 2, 8)
system.l2 = cache('256KiB', 8, 12, 20)
```

```
system.membus = SystemXBar()
system.l2bus = L2XBar()
```

cache(size, assoc, lat, mshrs) creates a gem5 Cache with the specified capacity, associativity, tag/data/response latency, and number of miss-status holding registers. Both L1 caches connect to the L2 crossbar, and L2 connects to main memory. A change in MSHRs can alter the number of outstanding misses, even if total bytes remain similar.

ANALYZE If the active x region is 8 KiB but the 32 KiB L1 data cache is shared with values, indices, row pointers, and output, is an x miss rate of zero guaranteed? Explain.

Trace probe: after L2, before the system memory bus

```
system.monitor = CommMonitor()
system.monitor.trace = MemTraceProbe(
    trace_file='kernel_postL2_all.trc', trace_compress=False)
system.l2.mem_side = system.monitor.cpu_side_port
system.monitor.mem_side_port = system.membus.cpu_side_ports
```

CommMonitor observes transactions at a particular gem5 port boundary without itself changing the CSR algorithm. MemTraceProbe records request fields in protobuf format, including tick, command ID, physical address and size. Placing it after L2 means this is memory-facing traffic, not every CPU load/store.

The instruction cache also reaches the shared L2, so a post-L2 trace is not guaranteed to contain only x gathers or only data-cache misses.

Main-memory baseline and workload

```
system.mem_ctrl = MemCtrl()
system.mem_ctrl.dram = DDR4_2400_8x8()
system.mem_ctrl.dram.range = system.mem_ranges[0]
system.workload = SEWorkload.init_compatible(binary)
system.cpu.workload = Process(cmd=[binary] + args)
```

gem5 executes the statically linked x86 binary in system-call emulation (SE) mode. The memory attached to gem5 is DDR4 for BOTH locality experiments. Ramulator HBM2 is a later independent trace-replay experiment; replacing the trace replay does not retroactively change gem5's CPU timing.

6. gem5 workbegin/workend event control

Read the actual configuration driver

```
m5.instantiate()
start_event = m5.simulate()
assert start_event.getCause().strip().lower() == 'workbegin'
start_tick = m5.curTick()
m5.stats.reset()
end_event = m5.simulate()
assert end_event.getCause().strip().lower() == 'workend'
end_tick = m5.curTick()
m5.stats.dump()
exit_event = m5.simulate()
```

NOTE The exact generated script uses explicit RuntimeError checks instead of assert; the excerpt above keeps the control flow compact. Read gem5_kernel_roi.py in your output directory for the complete implementation.

First m5.simulate() runs initialization until the guest begins the ROI. The reset ensures subsequent counters represent the kernel interval. The second simulate() advances through the CSR accumulation until workend. The dump records ROI statistics. The final simulate() permits the reference-output verification and clean program exit.

The script saves roi_window.json with start/end ticks and computes the interval as end − start. One tick corresponds to 10^-12 simulated seconds in this configuration. Do not confuse simulator host runtime (e.g., approximately four minutes) with modeled CSR execution time (tens of milliseconds).

Run the two kernel-ROI experiments

Use the same set of dimensions for each run. These commands can be run in the foreground; each automatically performs source validation, native oracle, gem5, initial Ramulator replay and a drain audit.

```
"$PY" "$LAB/Week8_Kernel_Only_CSR_SpMV.py" --run \
  --week8-root "$ROOT" --gem5-root "$GEM5_ROOT" \
  --ramulator-root "$RAM_ROOT" --rows 32768 \
  --nnz-per-row 16 --cols 262144 --pattern scattered

"$PY" "$LAB/Week8_Kernel_Only_CSR_SpMV.py" --run \
  --week8-root "$ROOT" --gem5-root "$GEM5_ROOT" \
  --ramulator-root "$RAM_ROOT" --rows 32768 \
  --nnz-per-row 16 --cols 262144 --pattern clustered
```

Each run creates a new uniquely named directory under $ROOT/results/kernel_roi/. It does not overwrite a prior experiment. Students should avoid starting multiple heavy gem5 runs simultaneously on a shared machine unless the instructor permits it.

Check success and locate outputs

```
find "$ROOT/results/kernel_roi" -name kernel_profile_verification.json
find "$ROOT/results/kernel_roi" -name kernel_trace_manifest.json
find "$ROOT/results/kernel_roi" -name gem5_kernel_roi_stats.txt
```

Required lines in the runner output include W8_KERNEL_DDR4_DRAIN=PASS, W8_KERNEL_HBM2_DRAIN=PASS, and W8_KERNEL_ROI_PIPELINE=PASS. Also inspect logs/gem5_roi.log for BOTH W8_START_EVENT: workbegin and W8_END_EVENT: workend.

7. From protobuf packet stream to trace records

Select requests between the ROI tick boundaries

```
tick = int(pkt.tick)
if tick < start:
    outside_before += 1
    continue
if tick >= end:
    outside_after += 1
    continue
inside += 1
```

The filter uses each request's recorded gem5 timestamp, not its completion timestamp. Requests before the ROI and after it are excluded. Timing is used to select records, but is NOT retained by Ramulator's LoadStoreTrace format.

Interpret gem5 packet command IDs

```
cmd = id_to_name.get(int(pkt.cmd), f'UNKNOWN_{int(pkt.cmd)}')
if cmd in READ_CMDS:
    op = 'LD'
elif cmd in WRITE_CMDS:
    op = 'ST'
else:
    skipped[cmd] += 1
    continue
addr = int(pkt.addr)
size = int(pkt.size)
f.write(f'{op} 0x{addr:x}\n')
```

The script reads the Command enumeration from the installed gem5 src/mem/packet.hh. For example, ReadSharedReq / ReadExReq represent read-type memory operations, and WritebackDirty represents write traffic. CleanEvict is not a data-transfer packet and is excluded. Unexpected unclassified commands cause a preflight failure.

The manifest separately records original request sizes, bytes, counts, command histogram, first/last accepted tick and SHA-256 of the generated trace. These fields are essential because the LD/ST file itself contains only operation and address.

ANALYZE Why would mapping every command labeled “unknown” by an older packet decoder to LD corrupt a DRAM experiment?

8. Validate the trace before DRAM simulation

Inspect the manifest, not only the request file

```
# Example format; addresses are actual captured physical addresses.
LD 0x5700
LD 0x1f3dc0
ST 0x1f3d80
```

The script uses an LD/ST trace with one operation and an address per line; the file does not include original byte size or interarrival interval. All accepted requests in the verified large experiments were 64 B, which is recorded in size_histogram_bytes.

```
# In a student's scattered experiment directory:
cat "$RUN_DIR/kernel_trace_manifest.json"
head -n 8 "$RUN_DIR/spmv_roi_postL2.trace"
wc -l "$RUN_DIR/spmv_roi_postL2.trace"
```

Set RUN_DIR to the directory printed as W8_KERNEL_OUTPUT_DIR by the runner. The row count in wc -l must equal read + write in the manifest. A trace is invalid if any request size is unexpectedly absent, negative, or unclassified.

What is being measured?

Quantity	Meaning	Not equivalent to
gem5 ROI ticks	Modeled CPU+cache+memory time within CSR loop	Physical server wall time
Post-L2 requests	Packets observed between L2 and memory bus	All x loads or all CPU instructions
Trace byte count	Sum of original gem5 packet.size values	Exactly measured off-chip DRAM energy
Ramulator cycles	DRAM controller modeled cycles for replay	gem5 CPU cycles
Ramulator throughput	Simulated controller transfer estimate	End-to-end SpMV throughput

9. How Ramulator 2.1 replays this trace

Frontend, DRAM device, scheduler and controller

```
frontend = ramulator.frontend.LoadStoreTrace(
    clock_ratio=1, path=str(trace_path))
# DDR4 case
DRAM = ramulator.dram.DDR4(org_preset='DDR4_8Gb_x8',
                           timing_preset='DDR4_2400R', rank=1)
controller = ramulator.controller.GenericDDR

# HBM2 case
DRAM = ramulator.dram.HBM2(org_preset='HBM2_2Gb',
                           timing_preset='HBM2_2000Mbps')
controller = ramulator.controller.HBM12
```

LoadStoreTrace presents the stored request sequence to the memory system, attempting requests at a normalized frontend rate. DDR4_2400R and HBM2_2000Mbps select different timing and organization models. GenericDDR and HBM12 implement controller behavior for their respective memory types.

```
ctrl = controller(dram=DRAM,
    scheduler=ramulator.scheduler.FRFCFS(),
    refresh_manager=ramulator.refresh_manager.AllBank(),
    row_policy=ramulator.row_policy.Open(),
    addr_mapper=ramulator.addr_mapper.RoBaRaCoCh())
```

FRFCFS prioritizes ready commands with a preference for row hits under its policy. AllBank handles refresh scheduling. Open keeps rows open until other constraints or decisions close them. RoBaRaCoCh maps physical address bits to DRAM row/bank/rank/column/channel structure; an address-map change can alter row locality even with identical requests.

```
mem = ramulator.memory_system.GenericDRAM(
    clock_ratio=1, controllers=[ctrl],
    channel_mapper=ramulator.channel_mapper.CacheLineInterleave())
sim = ramulator.Simulation(frontend, mem)
sim.run()
stats_yaml = sim.stats_yaml
sim.finalize()
```

One controller is configured for each backend. The simulation runs the entire trace, captures YAML statistics, and finalizes. Both frontend and memory clock ratios are set to 1 for this normalized teaching replay; they do not recreate gem5 request injection timing.

Important: request completion is locally patched

```
// Extract from instructor's Ramulator completion patch
req.callback = [this](Request&) {
    --m_w8_outstanding;
    ++m_w8_completed;
};
...
bool done = m_trace_count == m_trace_length &&
            m_w8_outstanding == 0;
```

The original teaching problem terminated when the final request was submitted. The instructor-installed patch waits until callbacks report all accepted requests completed (or forwarded/coalesced). It also rejects duplicate callbacks and imposes a finite drain timeout. This does NOT repair the normalized arrival-rate model; it repairs end-of-replay accounting.

ANALYZE Why can “all requests submitted” and “all requests completed” correspond to different cycle counts? Give a queueing example.

10. Equalize the DRAM payload before comparing

Why a one-line-to-one-line replay is unfair

The original 64-byte gem5 post-L2 packet is treated as a 64-byte DDR4 transaction, but the selected HBM2 configuration represents 32-byte pseudochannel transfers. Giving both backends one LD/ST line per source packet would compare different logical byte totals.

Exact transformation in the classroom payload script

```
a = int(parts[1], 0)
require(a >= 0 and a % 64 == 0,
        f'nonaligned 64-byte gem5 packet: {a:#x}')
fout.write(f'{parts[0]} 0x{a:x}\n')
fout.write(f'{parts[0]} 0x{a+32:x}\n')
```

For each aligned original 64-byte packet at address a, the HBM2 trace contains transfers covering [a,a+31] and [a+32,a+63]. DDR4 retains one 64-byte request at a. The transformation preserves read/write type, logical byte range, and source ordering, although HBM2 has twice as many frontend transactions.

```
ddr4_payload_bytes = 64 * original_records
hbm2_payload_bytes = 32 * (2 * original_records)
require(ddr4_payload_bytes == hbm2_payload_bytes,
        'payload mismatch')
```

The script checks SHA-256 of the original trace, verifies all gem5 requests have size 64 B and aligned addresses, runs both backends, and independently checks all callbacks and accepted/served request counts.

Run the corrected byte-equivalent experiment

```
"$PY" "$LAB/Week8_Classroom_Payload_Replay.py" --run \
  --week8-root "$ROOT" --ramulator-root "$RAM_ROOT"
```

This is a classroom adaptation of the instructor-verified payload script. The original assumed $HOME was /home/asinha. The classroom version takes explicit student-directory and installed-Ramulator paths and writes no data into the instructor's account.

```
cat "$ROOT/results/kernel_roi/payload_equivalent_summary.json"
```

Expected final marker: W8_PAYLOAD_EQUIVALENT=PASS. Each pattern reports callback completion, zero outstanding requests, command statistics, clock period assumptions, controller cycles and modeled replay time.

Convert controller cycles to physical time

For the selected presets, the experiment records assumed cycle periods of 833 ps for DDR4 and 1000 ps for HBM2. The script converts modeled cycles to milliseconds using cycles × tCK(ps) / 10^9. Check these timing presets if the simulator source or config is changed.

ANALYZE If DDR4 reports fewer controller cycles than HBM2 for a different trace, can you conclude it has lower physical replay time? Show the unit conversion required.

11. Verilator: cycle-by-cycle ready/valid behavior

What the RTL does (and does not) implement

The provided SystemVerilog file models a small accumulator with four signed output registers. It receives one precomputed product and row ID per accepted handshake. It is not an RTL CSR decoder, HBM interface, DDR4 controller, or complete hardware SpMV accelerator.

```
assign s_ready = rst_n && !done && accept_enable;
always_ff @(posedge clk) begin
  if (!rst_n) begin
    y0 <= 0; y1 <= 0; y2 <= 0; y3 <= 0;
    accepted <= 0; done <= 0;
  end else if (s_valid && s_ready) begin
    case (row_id)
      2'd0: y0 <= y0 + product;
      2'd1: y1 <= y1 + product;
      2'd2: y2 <= y2 + product;
      2'd3: y3 <= y3 + product;
```

```
    endcase
    accepted <= accepted + 3'd1;
    if (accepted == 3'd5) done <= 1;
  end
end
```

s_valid means the producer offers a stable tuple (row_id, product); s_ready means the consumer can accept it. Transfer occurs only at a rising edge with both high. Nonblocking assignments update registers on the simulated clock event, preserving synchronous semantics. The accepted counter reaches six; done prevents further acceptance.

Testbench injects deliberate stalls

```
product = VALUES[k] * X[COL_IDX[k]];
s_valid = 1;
accept_enable = 0;
repeat ((k % 3) + 1) @(negedge clk);
accept_enable = 1;
```

Before each handshake, the testbench holds valid and the payload constant while ready is low for 1–3 cycles. It then enables acceptance. The final test demands exactly six contributions and outputs [6, 6, −1, 15]. This tests stalling and signed accumulation but does not test arbitration across multiple simultaneous inputs.

Build and execute with the installed Verilator

```
mkdir -p "$ROOT/results/verilator_obj"
verilator --binary --timing --assert -j 2 --top-module tb_spmv \
  --Mdir "$ROOT/results/verilator_obj" \
  "$LAB/Week8_Verilator_CSR_Protocol_tb.sv"
"$ROOT/results/verilator_obj/Vtb_spmv" | tee "$ROOT/results/verilator_run.log"
```

Expected marker: VERILATOR_SPMV_PROTOCOL=PASS count=6 y=[6,6,-1,15]. The teaching copy fixes a comment that the local Verilator previously misinterpreted as a directive; no RTL logic was changed.

ANALYZE What would happen if the producer changed row_id while s_valid=1 and s_ready=0? Which assertion or testbench behavior would you add?

12. Python: report evidence without inventing results

Reconstruct metrics from generated files

Use Python to parse the final payload_equivalent_summary.json. This standard-library example prints evidence for both patterns, with exact units. Run it after the two gem5 experiments and the payload script have completed.

```
"$PY" - "$ROOT" <<'PYCODE'
import json, sys
from pathlib import Path
p = Path(sys.argv[1]) / 'results/kernel_roi/payload_equivalent_summary.json'
d = json.loads(p.read_text())
assert d['status'] == 'PASS'
for pat, v in d['results'].items():
    for tech in ('ddr4','hbm2'):
        c = v[tech]
        assert c['outstanding'] == 0
        hit = c['row_hits']; miss = c['row_misses']; conflict = c['row_conflicts']
        hit_pct = 100 * hit / (hit + miss + conflict)
        print(pat, tech, 'time_ms=', round(c['modeled_controller_time_ms'],6),
              'row_hit_pct=', round(hit_pct,2))
PYCODE
```

Explicit formulas for the analysis

Measurement	Formula / interpretation
gem5 kernel time	ROI ticks / 10^12 seconds
Locality runtime ratio	gem5 scattered ROI time / gem5 clustered ROI time
Request reduction factor	scattered post-L2 records / clustered post-L2 records
Logical transferred bytes	64 × post-L2 source request count (all observed requests 64 B)
Row-hit percentage	100 × row_hits / (row_hits + row_misses + row_conflicts)
Controller modeled time (ms)	controller_cycles × tCK(ps) / 10^9
DRAM replay ratio	DDR4 modeled replay ms / HBM2 modeled replay ms (same pattern)

Do not mix the two timing boundaries

gem5 reports CPU+cache+memory execution of the actual SpMV binary in its DDR4-configured system. Ramulator reports standalone memory-controller replay of a stream extracted from that gem5 execution. The Ramulator outputs cannot be substituted into the gem5 ROI time, because the memory models are not connected in closed loop and the timestamps are dropped.

ANALYZE Suppose the standalone HBM2 replay time decreases by 20%. Is it valid to claim a 20% faster SpMV kernel? Identify at least three missing dependencies.

13. Results record and analytical exercises

Before you read any measured values

1. Predict the relative L1/L2 behavior from an 8 KiB active x region versus a 1 MiB addressable x region; distinguish capacity from conflicts.
2. Estimate the lower-bound CSR value/index traffic per nonzero. Specify what this estimate excludes.
3. Predict whether the number of SpMV multiply-add operations changes between patterns.
4. Predict whether HBM2 must always win after equalizing transferred bytes, and state the assumptions needed.
5. Define a request-completion invariant for the Ramulator replay.

Complete the four-way evidence sheet

Metric	Scattered DDR4	Scattered HBM2	Clustered DDR4	Clustered HBM2
Logical bytes	_____	_____	_____	_____
Controller cycles	_____	_____	_____	_____
tCK (ps)	_____	_____	_____	_____
Modeled time (ms)	_____	_____	_____	_____
Row-hit %	_____	_____	_____	_____
Outstanding requests	_____	_____	_____	_____

Critical-reasoning questions

6. Compute gem5's scattered-to-clustered modeled kernel-time ratio. Is the difference completely explained by the post-L2 request-count ratio?
7. Compute the DRAM byte totals for both patterns and compare them to 524,288 nonzeros. Are these totals equal to pure value/index traffic?
8. Why might a trace with fewer requests have a higher reported controller throughput than one with more requests?
9. Why do row-hit rates differ between the DDR4 and HBM2 models even when their logical address ranges match?
10. How could a change in cache associativity, MSHRs, or input-vector working set falsify your explanation?
11. What source of experimental variation remains because the clustered and scattered matrices have different col_idx values?
12. Propose one follow-on experiment to study 2, 4, or 8 DRAM channels without treating simulated results as measured FPGA speedup.
13. Explain why replaying the same address sequence at one request per cycle is not equivalent to a cycle-faithful gem5→Ramulator interface.

What constitutes a defensible student claim?

For every main conclusion, submit one prediction, one observed numeric measure with units, one plausible mechanism, one competing explanation, and an experiment that would distinguish them. Unsupported claims such as "HBM2 is always 1.36× faster" receive no credit.

14. Instructor-verified reference dataset

These values were reported from the instructor's server1 runs on 5 November 2026. They are a reproducibility reference, not a guarantee of identical timing on every gem5 version/configuration and not physical hardware measurements.

Metric	Scattered	Clustered
Rows	32,768	32,768
Nonzeros	524,288	524,288
gem5 ROI simulated ms	47.379080	13.709754
Post-L2 requests	483,016	71,728
Post-L2 reads	480,956	69,776
Post-L2 writes	2,060	1,952
Logical bytes (64 B/record)	30,913,024	4,590,592

Reference interpretation: the clustered run has approximately 3.46× lower gem5 modeled kernel time and 6.73× fewer post-L2 request records. This is a workload-specific locality result, not a DRAM-technology replacement result.

Payload-equivalent Ramulator reference

Metric	Scattered DDR4	Scattered HBM2	Clustered DDR4	Clustered HBM2
Physical requests	483,016	966,032	71,728	143,456
Payload per request	64 B	32 B	64 B	32 B
Controller cycles	2,508,087	1,690,680	382,838	234,582
Modeled time (ms)	2.089236	1.690680	0.318904	0.234582
Row hits	122,443	534,459	69,412	137,473
Row misses	7,667	17,061	573	2,064
Row conflicts	352,906	414,512	1,743	3,919
Callbacks pending	0	0	0	0

Scattered DDR4/HBM2 replay-time ratio: 1.236×; clustered ratio: 1.359×. These are independent, single-channel, normalized-frontend controller-replay comparisons. They are not end-to-end SpMV speedups; byte-equivalent requests do not imply identical injection times or equivalent full-device bandwidth.

The comparison also does not account for HBM2's complete stack and multi-channel system topology. The address mapping and pseudochannel width differ, which can change row hits and conflicts even for equivalent logical traffic.

15. Submission checklist

- One short report (2–3 pages): workload contract, two pre-run predictions, two evidence tables, three reasoned conclusions, two limitations, and one proposed falsification experiment.
- The two kernel_profile_verification.json files, both kernel_trace_manifest.json files, and the two saved gem5 ROI statistics sections.
- payload_equivalent_summary.json plus each backend verification.json showing completed callbacks and zero outstanding requests.
- verilator_run.log with a PASS marker, and a short explanation of s_valid/s_ready behavior.
- Exact simulator versions/paths and the original Week 7 C++ SHA-256; do not upload the instructor's installed binaries or confidential files.

Do not submit enormous raw protobuf traces unless asked; retain them within your own authorized course workspace. Use meaningful filenames and include command lines so a reviewer can reproduce your results.

16. Troubleshooting and safeguards

Symptom	Likely issue / action
Permission denied under /home/asinha	Course group read/traverse access missing; contact TA/instructor. Never use instructor credentials.
gem5.opt "--version" unrecognized	Use git revision or gem5 normal run log; --version is not supported in this setup.
"Descriptors cannot be created directly"	Older generated protobuf module; the scripted parser sets PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python.
No r/w packets in decoded CSV	Post-L2 contains ReadSharedReq/ReadExReq/WritebackDirty. Use command IDs from packet.hh, not unknown→read guessing.
Ramulator import missing	Use the provided /home/asinha/tools/week8/venv/bin/python, subject to approved permissions.
Ramulator service counts incomplete	Do not use unpatched LoadStoreTrace. Instructor patch checks completion callbacks and timeout.
Verilator missing verilated_std.sv	System installation must include verilator-devel; instructor/system administrator fixes it.
Gem5 output older/partially missing	Check latest timestamped output directory and complete W8_* PASS markers.
gem5 slows under concurrent students	Instructor should sequence large runs or provide verified sample traces; do not increase -j indiscriminately.

17. Interpretation boundaries and source documentation

- Actual model: gem5 v25.1.0.0 X86TimingSimpleCPU at 2 GHz with 32 KiB L1I, 32 KiB L1D, 256 KiB L2, and DDR4 memory reference.
- ROI method: m5_work_begin/end markers around just the C++ CSR compute loop; other program activities run before/after.
- Trace method: request-timestamp filter of post-L2 protobuf packets; contains some traffic not uniquely attributable to x loads.
- DRAM sensitivity: Ramulator 2.1 LoadStoreTrace, locally patched to wait for completion; DDR4/HBM2 one modeled channel each; normalized injection.
- Payload matching: one 64 B DDR4 operation versus two consecutive 32 B HBM2 operations; logical bytes equivalent, transaction cadence different.
- RTL check: Verilator tests a six-input ready/valid accumulator matching Week 7 arithmetic; it is not a hardware SpMV accelerator.

Primary references (official tools)

gem5 M5ops documentation — https://www.gem5.org/documentation/general_docs/m5ops/

gem5 statistics reference — https://www.gem5.org/documentation/learning_gem5/part1/gem5_stats/

Ramulator 2.1 repository and Python frontend reference — https://github.com/CMU-SAFARI/ramulator2

Verilator command-line guide — https://verilator.org/guide/latest/exe_verilator.html

NOTE The professor-installed Ramulator callback patch and the classroom payload-workdir adaptation are local teaching modifications, not unmodified upstream APIs. Documentation verifies general interfaces; the exact experimental numbers are from the instructor's posted server1 results.